



VIT[®]
BHOPAL
www.vitbhopal.ac.in

CSE 2006 - Programming in Java

Course Type: LP

Credits: 3

Prepared by

Dr Komarasamy G

Associate Professor (Grade-2)

School of Computing Science and Engineering

VIT Bhopal University

Unit-2 Java Object-oriented Programming

- **Java OOP (Basics)**
- Java Class and Objects, Java Methods, Java Constructor, Java Strings, Java Access Modifiers, Java this keyword, Java final keyword, Java Recursion, Java instance of Operator, Java Single Class and Anonymous Class, Java enum Class
- **Java OOP (Inheritance & Polymorphism)**
- Java Inheritance, Java Method Overriding, Java super Keyword, **Abstract Class & Method, Java Interfaces, Java Polymorphism (overloading & overriding), Java Encapsulation**
- **Java OOP (Other types of classes)**
- Nested & Inner Class, Java Static Class, Java Anonymous Class, Java Singleton, Java enum Class, Java enum Constructor, Java enum String, Java Reflection

Abstract class in Java

- A class which is declared with the abstract keyword is known as an **abstract class in Java**.
- It can have **abstract and non-abstract methods** (method with the body).

Abstraction in Java

- **Abstraction** is a **process of hiding the implementation details** and showing only functionality to the user.
- Another way, it **shows only essential things** to the user and hides the internal details
- Example, **sending SMS** where you type the text and send the message.
- **You don't know the internal processing** about the message delivery.
- Abstraction lets you focus on what the object does instead of how it does it. Ways to achieve Abstraction

There are two ways to achieve abstraction in java

- **Abstract class and Interface.**

Abstract class in Java

- **Abstract class in Java**

- A class which is declared as **abstract** is known as an **abstract class**.
- It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember

- An abstract class must be declared with an **abstract keyword**.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have **constructors and static methods** also.
- It can have **final methods** which will force the subclass not to change the body of the method.

Example of abstract class

abstract class A{}

Abstract class in Java

Rules for Java Abstract class



Abstract class in Java

- **Abstract Method in Java**

- A method which is declared as abstract and does not have implementation is known as an **abstract method**.

Example of abstract method

```
abstract void printStatus();    //no method body and abstract
```

Abstract class in Java

Example of Abstract class that has an abstract method

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
abstract class Bike
{
    abstract void run();
}
class Honda4 extends Bike{
void run()
    {System.out.println("running safely");}
public static void main(String args[]){
    Bike obj = new Honda4();
    obj.run();
}
}
```

Output:
running safely

Abstract class in Java

In this example, create the instance of Rectangle class, draw() method of Rectangle class will be invoked.

File: *TestAbstraction1.java*

```
abstract class Shape
{ abstract void draw(); }
//In real scenario, implementation is provided by others i.e. unknown by end user
class Rectangle extends Shape{
void draw(){System.out.println("drawing rectangle");}
}
class Circle1 extends Shape
{ void draw(){System.out.println("drawing circle");} }
```

Output:
drawing circle

//In real scenario, method is called by programmer or user

```
class TestAbstraction1{
public static void main(String args[]){
Shape s=new Circle1();
//In a real scenario, object is provided through method, e.g., getShape() method
s.draw();
}
}
```

Abstract class in Java

Another example of Abstract class in java

File: *TestBank.java*

```
abstract class Bank
{   abstract int getRateOfInterest();   }
class SBI extends Bank{
int getRateOfInterest(){return 7;}
}
class PNB extends Bank{
int getRateOfInterest(){return 8;}
}
```

Output:

Rate of Interest is: 7 %

Rate of Interest is: 8 %

```
class TestBank{
public static void main(String args[]){
Bank b;
b=new SBI();
System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");
b=new PNB();
System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");
}}
```

Abstract class in Java

Abstract class having constructor, data member and methods

An abstract class can have a data member, abstract method, method body (non-abstract method), constructor, and even main() method.

File: **TestAbstraction2.java**

//Example of an abstract class that has abstract and non-abstract methods

```
abstract class Bike{  
    Bike() {System.out.println("bike is created");}  
    abstract void run();  
    void changeGear() {System.out.println("gear changed");} }
```

//Creating a Child class which inherits Abstract class

```
class Honda extends Bike{  
    void run(){System.out.println("running safely..");} }
```

//Creating a Test class which calls abstract and non-abstract methods

```
class TestAbstraction2{  
    public static void main(String args[]){  
        Bike obj = new Honda();  
        obj.run();  
        obj.changeGear();  
    } }
```

Output:

```
bike is created  
running safely..  
gear changed
```

Abstract class in Java

Rule: If there is an abstract method in a class, that class must be abstract.

```
class Bike12
{
    abstract void run();
}
```

Output:

compile time error

Rule: If you are extending an abstract class that has an abstract method, you must either provide the implementation of the method or make this class abstract.

// A Simple demonstration of abstract.

```
abstract class A {  
    abstract void callme();  
    // concrete methods are still allowed in abstract classes  
    void callmetoo() {  
        System.out.println("This is a concrete method.");  
    }  
}
```

```
class B extends A {  
    void callme() {  
        System.out.println("B's implementation of callme.");  
    }  
}
```

```
class AbstractDemo {  
    public static void main(String args[]) {  
        B b = new B();  
        b.callme();  
        b.callmetoo();  
    }  
}
```

Output:

B's implementation of callme
This is a concrete method

Abstract class in Java

Abstract class in Java references:

https://youtu.be/p_4Dyfplqkw

Edureka: <https://youtu.be/5X0Y--92pMI>

Tutorial point: <https://youtu.be/vkYJFrTrh5M>

Interfaces in Java

- An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.
- The interface in Java is a **mechanism to achieve abstraction**. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.
- In other words, you can say that interfaces can have **abstract methods** and **variables**. It cannot have a method body.
- Java Interface also **represents the IS-A relationship**.
- It cannot be instantiated just like the abstract class.
- Since Java 8, we can have **default and static methods** in an interface.
- Since Java 9, we can have **private methods** in an interface.

Why use Java interface?

1. It is used to achieve abstraction.
2. Can support the functionality of **multiple inheritance**.
3. It can be used to achieve loose coupling.

Interfaces in Java

How to declare an interface?

- An interface is declared by using the **interface keyword**.
- It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.
- A **class that implements** an interface must implement all the methods declared in the interface.

Syntax:

```
interface <interface_name>
{
    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

Interface in Java:

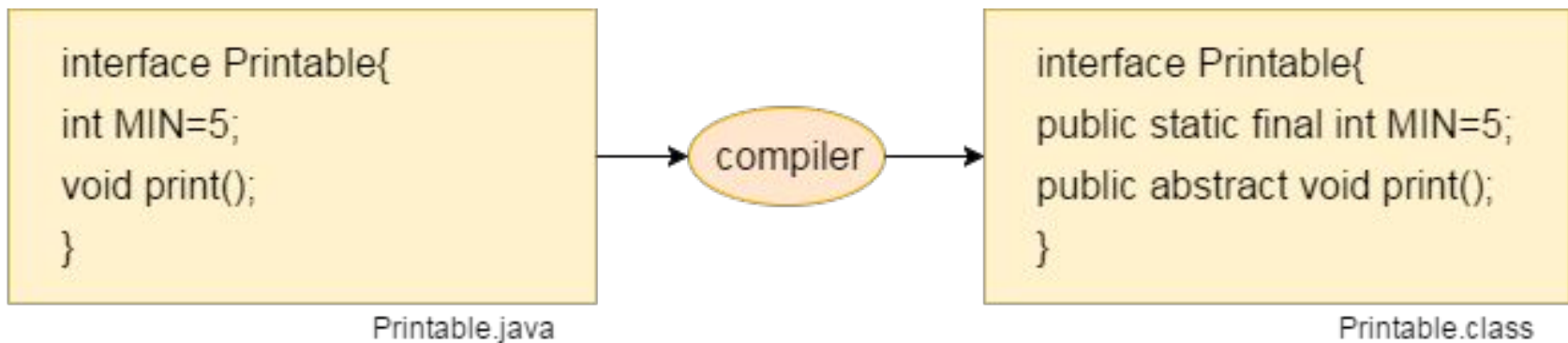
<https://www.javatpoint.com/interface-in-java>

https://youtu.be/jJ8L3SeFy_E

Interfaces in Java

Internal addition by the compiler

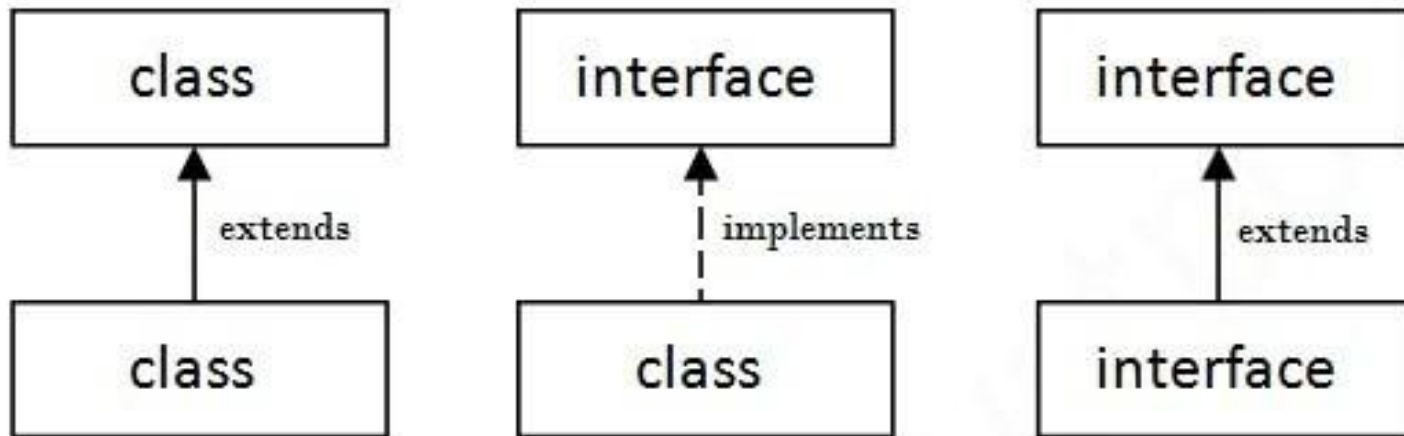
- The Java compiler adds **public** and **abstract** keywords before the interface method. Moreover, it adds **public**, **static** and **final** keywords before data members.
- In other words, Interface fields are **public**, **static** and **final** by default, and the methods are **public** and **abstract**.



Interfaces in Java

The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



Interfaces in Java

Java Interface Example

In this example, the Printable interface has only one method, and its implementation is provided in the A6 class.

```
interface printable
{
void print();
}
class A6 implements printable{
public void print()
{System.out.println("Hello");}

public static void main(String args[]){
A6 obj = new A6();
obj.print();
}
}
```

Output:
Hello

Interfaces in Java

Java Interface Example: Drawable

- In this example, the Drawable interface has only one method.
- Its implementation is provided by Rectangle and Circle classes.
- In a real scenario, an interface is defined by someone else, but its implementation is provided by different implementation providers.
- Moreover, it is used by someone else. The implementation part is hidden by the user who uses the interface.

Interfaces in Java

Java Interface Example: Drawable

File: *TestInterface1.java*

//Interface declaration: by first user

```
interface Drawable{  
void draw();  
}
```

//Implementation: by second user

```
class Rectangle implements Drawable{  
public void draw(){System.out.println("drawing rectangle");}  
}  
  
class Circle implements Drawable{  
public void draw(){System.out.println("drawing circle");}  
}
```

//Using interface: by third user

```
class TestInterface1  
{  
public static void main(String args[]){  
    Drawable d=new Circle();  
    //In real scenario, object is provided by method e.g. getDrawable()  
    d.draw();  
}}
```

Output:
drawing circle

Interfaces in Java

Java Interface Example: Bank

Let's see another example of java interface which provides the implementation of Bank interface.

File: *TestInterface2.java*

```
interface Bank{  
    float rateOfInterest();  
}  
  
class SBI implements Bank{  
    public float rateOfInterest(){return 9.15f;}  
}  
  
class PNB implements Bank{  
    public float rateOfInterest(){return 9.7f;}  
}  
  
class TestInterface2{  
    public static void main(String[] args){  
        Bank b=new SBI();  
        System.out.println("ROI: "+b.rateOfInterest());  
    }  
}
```

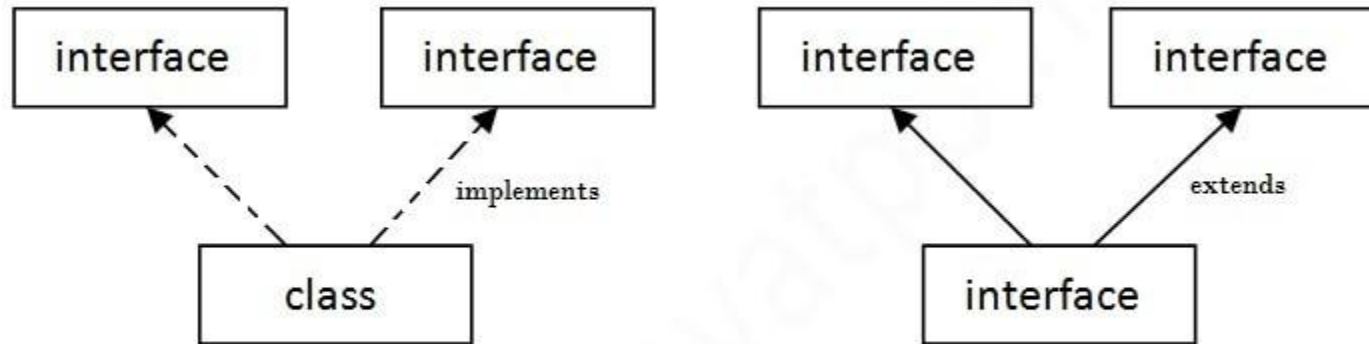
Output:

ROI: 9.15

Interfaces in Java

Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

Interfaces in Java

```
interface Printable
{ void print(); }
interface Showable
{ void show(); }
class A7 implements Printable, Showable
{
    public void print()
        {System.out.println("Hello");}
    public void show()
        {System.out.println("Welcome");}

    public static void main(String args[])
    {
        A7 obj = new A7();
        obj.print();
        obj.show();
    } }
```

Output:
Hello
Welcome

Interfaces in Java

Q) Multiple inheritance is not supported through class in java, but it is possible by an interface, why?

As we have explained in the inheritance chapter, multiple inheritance is not supported in the case of class because of ambiguity. However, it is supported in case of an interface because there is no ambiguity. It is because its implementation is provided by the implementation class. For example:

```
interface Printable{
```

```
void print();
```

```
}
```

```
interface Showable{
```

```
void print();
```

```
}
```

```
class TestInterface3 implements Printable, Showable
```

```
{
```

```
public void print() {System.out.println("Hello");}
```

```
public static void main(String args[]){
```

```
TestInterface3 obj = new TestInterface3();
```

```
obj.print();
```

```
} }
```

Printable and Showable interface have same methods but its implementation is provided by class TestInterface3, so there is no ambiguity.

Output:

Hello

Interfaces in Java

Interface inheritance

A class implements an interface, but one interface extends another interface.

```
interface Printable{  
    void print();  
}  
interface Showable extends Printable{  
    void show();  
}  
class TestInterface4 implements Showable{  
    public void print(){System.out.println("Hello");}  
    public void show(){System.out.println("Welcome");}  
  
    public static void main(String args[]){  
        TestInterface4 obj = new TestInterface4();  
        obj.print();  
        obj.show();  
    }  
}
```

Output:

Hello

Welcome

Interfaces in Java

Java 8 Static Method in Interface

File: *TestInterfaceStatic.java*

```
interface Drawable{  
    void draw();  
    static int cube(int x){return x*x*x;}  
}  
class Rectangle implements Drawable{  
    public void draw(){System.out.println("drawing rectangle");}  
}
```

```
class TestInterfaceStatic{  
    public static void main(String args[]){  
        Drawable d=new Rectangle();  
        d.draw();  
        System.out.println(Drawable.cube(3));  
    }  
}
```

Output:

```
drawing rectangle  
27
```

Interfaces in Java

Q) What is marker or tagged interface?

An interface which has **no member** is known as a marker or tagged interface, for example, Serializable, Cloneable, Remote, etc. They are used to provide some essential information to the JVM so that JVM may perform some useful operation.

//How Serializable interface is written?

```
public interface Serializable  
{ }
```

Nested Interface in Java

Note: An interface can have another interface which is known as a nested interface. We will learn it in detail in the nested classes chapter.

For example:

```
interface printable{  
    void print();  
    interface MessagePrintable{  
        void msg();  
    }  
}
```

Interfaces in Java

Can we define a class inside the interface?

Yes, if we define a class inside the interface, the Java compiler creates a static nested class.

Let's see how can we define a class within the interface:

```
interface M
```

```
{
```

```
    class A{}
```

```
}
```

Interfaces in Java

Example of nested interface which is declared within the interface

In this example, we will learn how to declare the nested interface and how we can access it.

TestNestedInterface1.java

```
interface Showable{  
    void show();  
    interface Message{  
        void msg();  
    }  
}
```

```
class TestNestedInterface1 implements Showable.Message{  
    public void msg(){System.out.println("Hello nested interface");}
```

```
    public static void main(String args[]){  
        Showable.Message message=new TestNestedInterface1();//upcasting here  
        message.msg();  
    }  
}
```

Output:

hello nested interface

Interfaces in Java

Example of nested interface which is declared within the class

Let's see how we can define an interface inside the class and how we can access it.

TestNestedInterface2.java

```
class A{  
    interface Message{  
        void msg();  
    }  
}
```

Output:

hello nested interface

```
class TestNestedInterface2 implements A.Message{  
    public void msg(){System.out.println("Hello nested interface");}
```

```
    public static void main(String args[]){  
        A.Message message=new TestNestedInterface2();//upcasting here  
        message.msg();  
    }  
}
```

Difference between abstract class and interface

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
9)Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>

Example of abstract class and interface in Java

//Creating interface that has 4 methods

```
interface A{  
void a();//bydefault, public and abstract  
void b();  
void c();  
void d();  
}
```

//Creating abstract class that provides the implementation of one method of A interface

```
abstract class B implements A{  
public void c()  
{System.out.println("I am C");}  
}
```

//Creating subclass of abstract class, now we need to provide the implementation of rest of the methods

```
class M extends B{  
public void a(){System.out.println("I am a");}  
public void b(){System.out.println("I am b");}  
public void d(){System.out.println("I am d");}  
}
```

//Creating a test class that calls the methods of A interface

```
class Test5{  
public static void main(String args[])  
{  
A a=new M();  
a.a();  
a.b();  
a.c();  
a.d();  
}}
```

Output:

```
I am a  
I am b  
I am c  
I am d
```

Applying Interfaces

interface IntStack **// Define an integer stack interface**

```
{  
    void push(int item);    // store an item  
    int pop();              // retrieve an item  
}
```

// An implementation of IntStack that uses fixed storage.

class FixedStack implements IntStack {

```
    private int stck[];
```

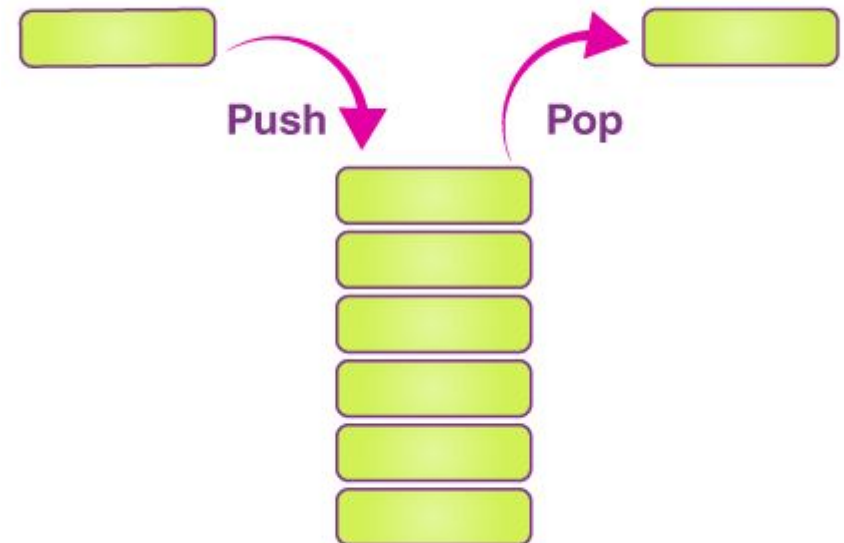
```
    private int tos;
```

```
    // allocate and initialize stack
```

```
    FixedStack(int size) {
```

```
        stck = new int[size];
```

```
        tos = -1;    }
```

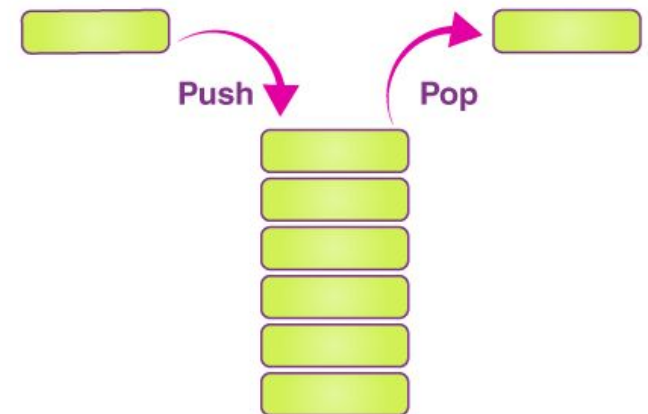


// Push an item onto the stack

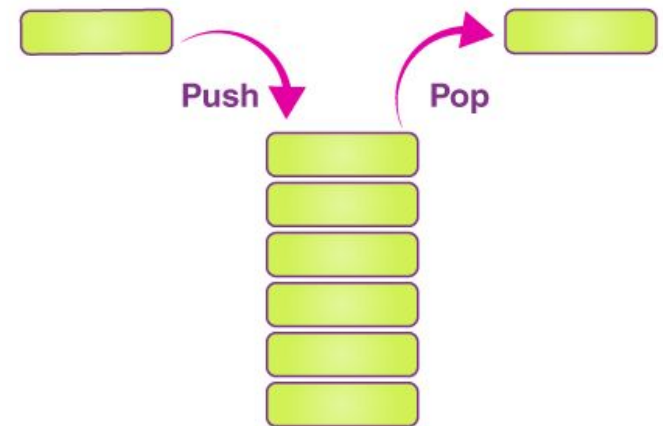
```
public void push(int item) {  
    if(tos==stck.length-1)    // use length member  
        System.out.println("Stack is full.");  
    else  
        stck[++tos] = item;  
}
```

// Pop an item from the stack

```
public int pop() {  
    if(tos < 0) {  
        System.out.println("Stack underflow.");  
        return 0;    }  
    else  
        return stck[tos--];    }    }
```



```
class IFTest {  
    public static void main(String args[]) {  
        FixedStack mystack1 = new FixedStack(5);  
        FixedStack mystack2 = new FixedStack(8);  
        // push some numbers onto the stack  
        for(int i=0; i<5; i++) mystack1.push(i);  
        for(int i=0; i<8; i++) mystack2.push(i);  
        // pop those numbers off the stack  
        System.out.println("Stack in mystack1:");  
        for(int i=0; i<5; i++)  
            System.out.println(mystack1.pop());  
        System.out.println("Stack in mystack2:");  
        for(int i=0; i<8; i++)  
            System.out.println(mystack2.pop());  
    }  
}
```



Interfaces can be Extended

- One interface can inherit another by use of the keyword **extends**.
- When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

```
// One interface can extend another.
interface A {
    void meth1();
    void meth2();
}

// B now includes meth1() and meth2() -- it adds meth3().
interface B extends A {
    void meth3();
}

// This class must implement all of A and B
class MyClass implements B {
    public void meth1() {
        System.out.println("Implement meth1().");
    }

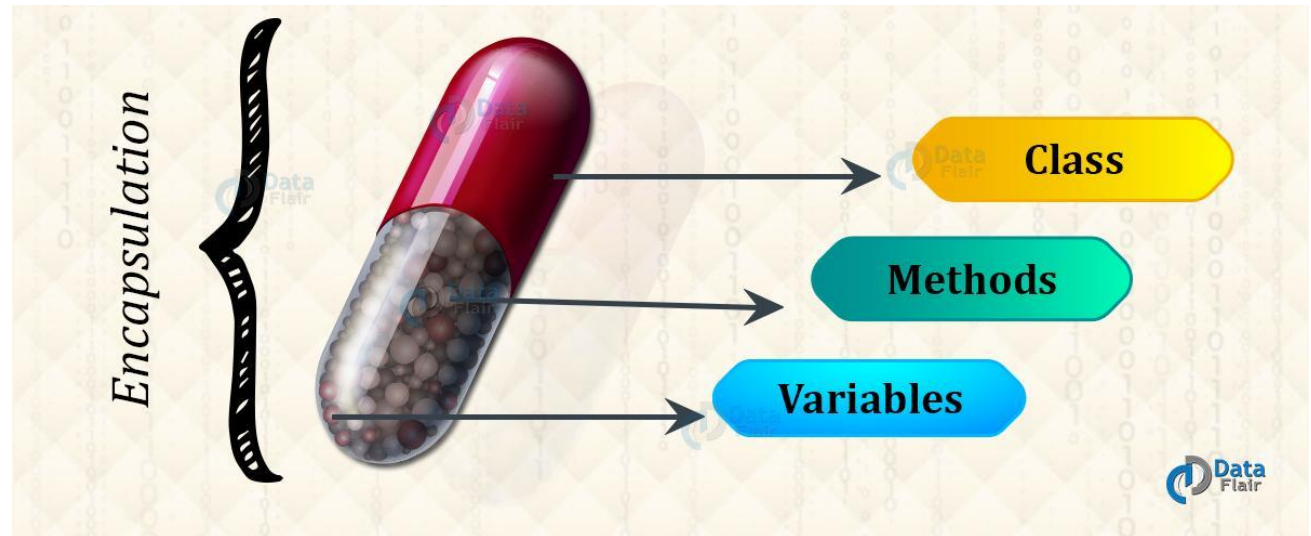
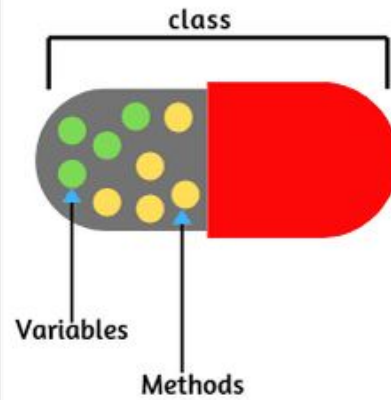
    public void meth2() {
        System.out.println("Implement meth2().");
    }

    public void meth3() {
        System.out.println("Implement meth3().");
    }
}
```

```
class IFExtend {  
    public static void main(String arg[]) {  
        MyClass ob = new MyClass();  
  
        ob.meth1();  
        ob.meth2();  
        ob.meth3();  
    }  
}
```

Encapsulation in Java

```
class  
{  
  
    data members  
    +  
    methods (behavior)  
}
```



Encapsulation

- Encapsulation is **one of the four fundamental OOP concepts**. The other three are inheritance, polymorphism, and abstraction.
- Encapsulation in Java is a mechanism of **wrapping the data (variables) and code acting on the data (methods) together as a single unit**.
- In encapsulation, the variables of a class will be hidden from other classes, and can be accessed only through the methods of their current class. Therefore, it is also known as data hiding.

To achieve encapsulation in Java –

- Declare the variables of a class as private.
- Provide public setter and getter methods to modify and view the variables values.

Benefits of Encapsulation

- The fields of a class can be **made read-only or write-only**.
- A class can have total control over what is **stored in its fields**.

Encapsulation

```
/* File name : EncapTest.java */
public class EncapTest {
    private String name;
    private String idNum;
    private int age;

    public int getAge() {
        return age;
    }

    public String getName() {
        return name;
    }

    public String getIdNum() {
        return idNum;
    }

    public void setAge( int newAge) {
        age = newAge;
    }

    public void setName(String newName) {
        name = newName;
    }

    public void setIdNum( String newId) {
        idNum = newId;
    }
}
```

Encapsulation

```
/* File name : RunEncap.java */  
public class RunEncap {  
  
    public static void main(String args[]) {  
        EncapTest encap = new EncapTest();  
        encap.setName("James");  
        encap.setAge(20);  
        encap.setIdNum("12343ms");  
  
        System.out.print("Name : " + encap.getName() + " Age : " + encap.getAge());  
    }  
}
```

Output

Name : James Age : 20

Encapsulation

```
class EncapsulationDemo
{
    private int ssn;
    private String empName;
    private int empAge; //Getter and Setter methods
    public int getEmpSSN()
    { return ssn; }
    public String getEmpName()
        { return empName; }
    public int getEmpAge()
        { return empAge; }
    public void setEmpAge(int newValue)
        { empAge = newValue; }
    public void setEmpName(String newValue)
        { empName = newValue; }
    public void setEmpSSN(int newValue)
        { ssn = newValue; }
}
```

Encapsulation

```
public class EncapsTest
{
    public static void main(String args[])
    {
        EncapsulationDemo obj = new EncapsulationDemo();
        obj.setEmpName("Mario");
        obj.setEmpAge(32);
        obj.setEmpSSN(112233);
        System.out.println("Employee Name: " + obj.getEmpName());
        System.out.println("Employee SSN: " + obj.getEmpSSN());
        System.out.println("Employee Age: " + obj.getEmpAge());
    }
}
```

Output:

```
Employee Name: Mario
Employee SSN: 112233
Employee Age: 32
```

In above example all the three data members (or data fields) are private(see: Access Modifiers in Java) which cannot be accessed directly. These fields can be accessed via public methods only. Fields empName, ssn and empAge are made hidden data fields using **encapsulation technique of OOPs.**